



LeetCode 344: Reverse String

1. Problem Title & Link

Title: LeetCode 344: Reverse String

Link: <https://leetcode.com/problems/reverse-string/>

2. Problem Statement (Short Summary)

Write a function that reverses a character array **in-place**.

You are given:

`s = ['h','e','l','l','o']`

You must modify the same array to become:

`['o','l','l','e','h']`

You must:

Use $O(1)$ extra space

Example

Input:

`s = ["h","e","l","l","o"]`

Output:

`["o","l","l","e","h"]`

3. Examples (Input → Output)

Example 1

Input:

`["h","e","l","l","o"]`

Output:

`["o","l","l","e","h"]`

Example 2

Input:

`["H","a","n","n","a","h"]`

Output:

`["h","a","n","n","a","H"]`

Example 3

Input:

`["a"]`

Output:



```
["a"]
```

Example 4

Input:

```
["a","b"]
```

Output:

```
["b","a"]
```

4. Constraints $1 \leq s.length \leq 10^5$

Important:

Must modify array in-place.

Do not create another array.

5. Core Concept (Pattern / Topic)**Two Pointers**

Secondary Topics:

- In-Place Swapping
- Array Manipulation

6. Thought Process (Step-by-Step Explanation)**Brute Force Idea**

Create a new reversed array.

Example:

```
["h","e","l","l","o"]
```

Create:

```
["o","l","l","e","h"]
```

Copy back.

Works.

Problem

Extra memory required.

Space = $O(n)$

Question specifically asks:

In-place



Better Observation

To reverse an array:

First $\leftrightarrow$ Last

Second $\leftrightarrow$ Second Last

Third $\leftrightarrow$ Third Last

We don't need another array.

Just swap.

Example

h e l l o

↑ ↑

Swap:

o e l l h

Move inward:

o e l l h

↑ ↑

Swap:

o l l e h

Done.

Key Insight

Use:

left pointer

right pointer

Swap until they meet.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Initial:
h e l l o
↑       ↑
L       R
Swap:
o e l l h
   ↑   ↑
   L   R
Swap:
```



```
o l l e h
  ↑
Pointers cross.
Finished.
```

8. Pseudocode (Language Independent)

```
left = 0

right = n - 1

while left < right

    swap(left, right)

    left++

    right--

return
```

9. Code Implementation

Python

```
class Solution:
    def reverseString(self, s) -> None:

        left = 0
        right = len(s) - 1

        while left < right:

            s[left], s[right] = \
                s[right], s[left]

            left += 1
            right -= 1
```

Java

```
class Solution {

    public void reverseString(char[] s) {

        int left = 0;
        int right = s.length - 1;

        while(left < right) {
```



```
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;

        left++;
        right--;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Each element is visited at most once.

Space Complexity

$O(1)$

Only two pointers are used.

11. Common Mistakes / Edge Cases

Mistake 1

Creating a new array.

`s[::-1]`

Works.

But not truly in-place.

Mistake 2

Looping until:

`left <= right`

Not wrong, but causes unnecessary middle swap for odd lengths.

Preferred:

`left < right`

Mistake 3

Forgetting pointer updates.

Can cause:

Infinite Loop



Edge Cases

Single Character

Input:

["a"]

Output:

["a"]

Two Characters

Input:

["a","b"]

Output:

["b","a"]

Palindrome

Input:

["m","a","d","a","m"]

Output:

["m","a","d","a","m"]

Same after reversal.

12. Detailed Dry Run

Input:

["h","e","l","l","o"]

Left	Right	Array
0	4	h e l l o
Swap		o e l l h
1	3	o e l l h
Swap		o l l e h
2	2	Stop

Final:

["o","l","l","e","h"]

13. Common Use Cases (Real-Life / Interview)

Text Processing



Reverse words or characters.

DNA Sequence Analysis

Reverse sequence operations.

Data Transformation

Reverse arrays efficiently.

Interview Purpose

Tests:

Two Pointers

In-Place Modification

Swapping

14. Common Traps (Important!)

Trap 1

Using built-in reverse functions immediately.

Interviewers usually want the logic.

Trap 2

Creating extra arrays.

Violates:

$O(1)$ space

Trap 3

Incorrect pointer movement.

Example:

left += 1

but forgetting:

right -= 1

15. Builds To (Related LeetCode Problems)

LC 344 - Reverse String



LC 125 - Valid Palindrome



LC 680 - Valid Palindrome II



LC 557 - Reverse Words in a String III



LC 151 - Reverse Words in a String

These strengthen Two Pointer string techniques.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Extra Array	$O(n)$	$O(n)$	Fair
Built-In Reverse	$O(n)$	$O(1)/O(n)$	Easy
Two Pointers	$O(n)$	$O(1)$	Best

Approach 1: Extra Array

```
temp = s[::-1]
```

Simple.

Uses extra memory.

Approach 2: Built-In

Python:

```
s.reverse()
```

Easy.

Doesn't show understanding.

Approach 3: Two Pointers (Chosen)

Most efficient.

Most common interview solution.

17. Why This Solution Works (Short Intuition)

The first character belongs at the last position, and the last character belongs at the first position.

By swapping symmetric positions and moving inward, the entire array becomes reversed without requiring additional memory.

18. Variations / Follow-Up Questions

Follow-Up 1

Reverse only vowels.

Related:



LC 345

Reverse Vowels of a String

Follow-Up 2

Reverse every word individually.

Related:

LC 557

Reverse Words in a String III

Follow-Up 3

Reverse words order.

Related:

LC 151

Reverse Words in a String

Follow-Up 4

Check whether a string is a palindrome.

Related:

LC 125

Valid Palindrome

Interview Takeaway

This is one of the most important **Two Pointer** problems.

Core idea:

Start from both ends

Swap

Move inward

Whenever you hear:

Reverse

Mirror

Symmetric positions

In-place modification

think:

Two Pointers

This pattern appears repeatedly in string, array, and palindrome interview problems.